

Самостоятельная работа №1:

Реализация алгоритма AES

1. Цель самостоятельной работы

Целью данной самостоятельной работы является изучение и практическая реализация алгоритма симметричного шифрования AES (Advanced Encryption Standard) с использованием языка программирования Python. В процессе выполнения работы студенты научатся шифровать и дешифровать данные, используя библиотеку `pycryptodome`, а также узнают основные принципы работы AES.

2. Подробное техническое задание к самостоятельно работе

1. Установить библиотеку `pycryptodome` для работы с криптографическими алгоритмами в Python.
2. Создать скрипт на языке Python, который будет реализовывать следующие функции:
 - Генерация случайного ключа для алгоритма AES.
 - Шифрование заданного текста с использованием AES в режиме CBC (Cipher Block Chaining).
 - Дешифрование зашифрованного текста.
3. Обеспечить корректное выравнивание (padding) данных для шифрования, используя PKCS7.
4. Реализовать обработку ошибок и исключительных ситуаций.
5. Протестировать реализацию на нескольких примерах, убедившись в правильности шифрования и дешифрования.

3. Список рекомендаций по использованию библиотек, структур данных, паттернов проектирования

Библиотеки

- `pycryptodome`: Это библиотека для работы с криптографическими примитивами в Python. Установите её с помощью команды:

```
pip install pycryptodome
```

Структуры данных

- Используйте байтовые строки (bytes) для представления данных, которые будут шифроваться и дешифроваться.
- Используйте словари для хранения ключей и других параметров шифрования, если это необходимо.

Паттерны проектирования

- **Функциональный подход:** Разделите ваш код на небольшие функции, каждая из которых выполняет конкретную задачу (например, генерация ключа, шифрование, дешифрование).
- **Обработка исключений:** Используйте блоки `try-except` для обработки возможных ошибок при шифровании и дешифровании данных.

4. Формат отчета

Отчет по самостоятельно работе должен включать следующие разделы:

Введение

- Краткое описание цели и задач самостоятельной работы.

Теоретическая часть

- Описание алгоритма AES и его основных режимов работы.
- Объяснение используемых методов выравнивания (padding).

Практическая часть

- Описание используемых инструментов и библиотек.
- Исходный код программы с комментариями.
- Скриншоты или вывод результатов тестирования программы.

Заключение

- Выводы по результатам выполнения лабораторной работы.
- Описание возможных улучшений и доработок.

Приложения

- Дополнительные материалы, если таковые имеются (например, тестовые данные, дополнительный код).

Пример реализации в Python

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad

# Генерация случайного ключа для AES
key = get_random_bytes(16) # 16 байт для AES-128
iv = get_random_bytes(16) # 16 байт для IV

def encrypt(data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    padded_data = pad(data, AES.block_size)
    encrypted_data = cipher.encrypt(padded_data)
    return encrypted_data
```

```
def decrypt(encrypted_data, key, iv):
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_data = unpad(cipher.decrypt(encrypted_data), AES.block_size)
    return decrypted_data

# Тестирование реализации
plain_text = b'Hello, this is a test message!'
encrypted_text = encrypt(plain_text, key, iv)
decrypted_text = decrypt(encrypted_text, key, iv)

print(f'Original text: {plain_text}')
print(f'Encrypted text: {encrypted_text}')
print(f'Decrypted text: {decrypted_text}')
```

Соблюдайте структуру отчета и приводите результаты тестирования, чтобы продемонстрировать правильность работы вашей реализации.